ABSTRACT

This project describes the design of an automated mini conveyor belt whose DC motor speed is regulated using feedback from an optical sensor. An Arduino UNO measures pulses produced by a marked rotating pulley or encoder disc. The measured rotational speed is compared with a user-defined target. A proportional-integral (PI) control routine adjusts the pulse-width modulation (PWM) command sent to an L298N motor driver, which powers the DC motor. The feedback loop is intended to compensate for speed changes caused by load variation and other disturbances. The report includes the system design, component list, wiring plan, control equations, Arduino program, calibration and test procedure, and a results sheet to complete after physical or simulated testing.

Important project-status note: this document supplies a design and test procedure. No physical build, simulator run, screenshots, or measured results were provided with the brief. Therefore, the results section is clearly marked for recording actual observations rather than claiming tests that have not been performed.

1. INTRODUCTION

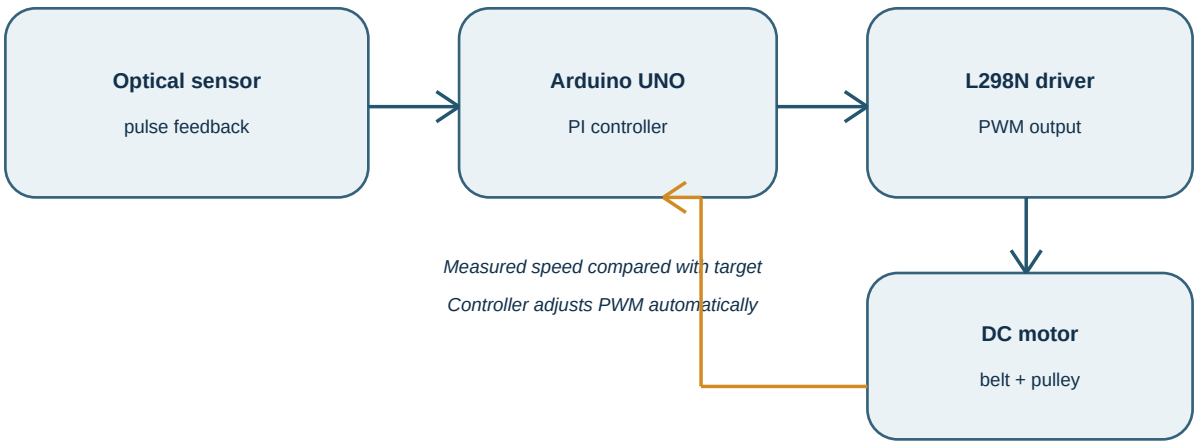
Conveyor belts are used to transport items between locations in manufacturing, packaging, sorting, and material-handling systems. If the motor speed changes when the load changes, items may move too slowly or too quickly. Open-loop control applies a fixed motor command and does not correct the speed automatically. Closed-loop control measures the actual speed and changes the motor command based on the difference between actual and desired speed.

In this project, an optical sensor detects repeated marks or slots on a rotating disc connected to the drive shaft or pulley. The Arduino calculates the speed from the pulses and updates the motor PWM. The L298N provides the motor-driving interface between the controller and DC motor.

2. OBJECTIVES

- Design a mini conveyor belt speed-control system.
- Drive the belt using a DC motor and an L298N motor driver.
- Use an optical proximity sensor to measure rotation or movement.
- Implement closed-loop speed regulation using sensor feedback.
- Automatically adjust PWM when measured speed differs from the target.
- Test operation at different loads and setpoints, and record actual performance.

3. SYSTEM OVERVIEW



The sensor produces pulses as the marked disc rotates. The Arduino counts these pulses during a fixed sampling interval and estimates RPM. The controller calculates the error between target RPM and measured RPM, then adjusts the PWM command. The L298N switches motor current according to the PWM signal. The motor rotates the pulley, which moves the belt. The sensor continuously measures the resulting motion, closing the feedback loop.

4. COMPONENTS REQUIRED

Component	Purpose / notes
Arduino UNO	Reads sensor pulses and runs the speed-control algorithm.
L298N dual H-bridge module	Drives the DC motor; ENA receives PWM. Use an appropriate motor supply.
DC geared motor	Provides the conveyor drive. Match voltage/current to the driver and supply.
Optical proximity / speed sensor	Detects a slotted disc or reflective marks; output must be compatible with Arduino logic.
Pulley, belt, frame, rollers	Mechanical conveyor assembly.
Encoder disc / marked wheel	Creates a known number of sensor pulses per shaft revolution.
External DC supply	Powers the motor; rating must suit the motor and driver.
Wires, mounting hardware, test load	Connections, secure mounting, and controlled testing.

5. CONNECTIONS / WIRING

Arduino UNO pin	Connect to	Notes
D5 (PWM)	L298N ENA	Remove ENA jumper if fitted so PWM can control speed.
D7	L298N IN1	Direction control.
D8	L298N IN2	Direction control.
D2 (interrupt)	Optical sensor OUT	Sensor must output one clean digital pulse per mark/slot.
5V	Sensor VCC, if rated for 5 V	Check the sensor's label/datasheet before powering it.
GND	Sensor GND and L298N GND	Controller, sensor and driver grounds must be common.
External motor supply +	L298N motor supply terminal	Use the motor's rated voltage; do not power the motor from an Arduino I/O pin.
Motor terminals	L298N OUT1 and OUT2	If direction is reversed, swap the motor wires or reverse IN1/IN2 logic.

Wiring and safety notes

• Disconnect power before changing wiring. Keep fingers, loose wires, and clothing away from the moving belt and pulley.
• Do not connect the motor directly to the Arduino. Use a suitable external motor supply and driver.
• The Arduino GND and L298N GND must be connected together for reliable control signals.
• Confirm the motor current is within the L298N module's practical rating; the L298N has a noticeable voltage drop and can become hot.
• If the sensor output is open-collector, use the pull-up recommended by its datasheet. The code below uses an interrupt on D2 and assumes a clean digital pulse.

6. WORKING PRINCIPLE

The system follows these steps:
1. A target speed is set in the program.
2. As the marked disc rotates, the optical sensor generates pulses.
3. The Arduino counts pulses over a sampling period.
4. Pulse count is converted to revolutions per minute (RPM).
5. The speed error is calculated: error = target RPM – measured RPM.
6. A PI controller calculates a new PWM command between 0 and 255.
7. The L298N changes the motor's average applied voltage, and the loop repeats.

7. CONTROL EQUATIONS

Let P be the number of pulses counted during a sampling interval Ts seconds, and N be the number of sensor pulses per shaft revolution.

Measured speed: RPM = (P / N) × (60 / Ts)
Speed error: e(k) = RPMtarget – RPMmeasured
PI control: u(k) = Kp × e(k) + Ki × Σ[e(k) × Ts]
PWM command: limited to the range 0–255.

If belt pulley diameter is D metres, the ideal belt speed is v = π × D × RPM / 60 metres per second. This assumes no belt slip and that the measured shaft is directly coupled to the drive pulley. The target RPM and pulley diameter should be calibrated to the actual assembly.

8. ARDUINO PROGRAM

The program below is for an Arduino UNO with an L298N and a digital optical speed sensor. Set PULSES_PER_REV to match the number of pulses generated per revolution. The starting PI gains are tuning values, not guaranteed final values for every motor. Test with the motor securely mounted and adjust gradually.

```
/*
  Automated Mini Conveyor Belt Speed Regulator
  Arduino UNO + L298N + optical speed sensor
  D2 = sensor pulse output (interrupt)
  D5 = ENA PWM, D7 = IN1, D8 = IN2
*/

const byte SENSOR_PIN = 2;
const byte ENA_PIN = 5;   // PWM pin
const byte IN1_PIN = 7;
const byte IN2_PIN = 8;

const unsigned long SAMPLE_MS = 500;
const float PULSES_PER_REV = 20.0; // CHANGE to match your encoder disc
const float TARGET_RPM = 60.0;     // desired shaft speed

// Initial PI tuning values. Tune carefully on the actual mechanism.
const float KP = 1.2;
const float KI = 0.8;

volatile unsigned long pulseCount = 0;
unsigned long lastSampleMs = 0;
float integralTerm = 0.0;
int pwmCommand = 100;

void sensorPulse() {
  pulseCount++;
}

void setup() {
  pinMode(SENSOR_PIN, INPUT_PULLUP);
  pinMode(ENA_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);

  // Forward direction; reverse IN1/IN2 if the belt moves backward.
  digitalWrite(IN1_PIN, HIGH);
  digitalWrite(IN2_PIN, LOW);

  Serial.begin(9600);
  attachInterrupt(digitalPinToInterrupt(SENSOR_PIN), sensorPulse, FALLING);

  analogWrite(ENA_PIN, pwmCommand);
```

```
    lastSampleMs = millis();
```

8. ARDUINO PROGRAM (continued)

```
  Serial.println("time_ms,target_rpm,measured_rpm,pwm,error");
}

void loop() {
  unsigned long now = millis();

  if (now - lastSampleMs >= SAMPLE_MS) {
    unsigned long elapsedMs = now - lastSampleMs;
    lastSampleMs = now;

    noInterrupts();
    unsigned long pulses = pulseCount;
    pulseCount = 0;
    interrupts();

    float elapsedSeconds = elapsedMs / 1000.0;
    float measuredRPM =
      ((float)pulses / PULSES_PER_REV) * (60.0 / elapsedSeconds);

    float error = TARGET_RPM - measuredRPM;

    // Discrete PI controller with anti-windup by clamping the integral.
    integralTerm += error * elapsedSeconds;
    if (integralTerm > 200.0) integralTerm = 200.0;
    if (integralTerm < -200.0) integralTerm = -200.0;

    float control = (KP * error) + (KI * integralTerm);
    pwmCommand += (int)control;

    if (pwmCommand > 255) pwmCommand = 255;
    if (pwmCommand < 0)   pwmCommand = 0;

    analogWrite(ENA_PIN, pwmCommand);

    Serial.print(now);
    Serial.print(',');
    Serial.print(TARGET_RPM, 1);
    Serial.print(',');
    Serial.print(measuredRPM, 1);
    Serial.print(',');
    Serial.print(pwmCommand);
    Serial.print(',');
    Serial.println(error, 1);
```

8. ARDUINO PROGRAM (continued)

```
  }
}
```

9. CALIBRATION AND TEST PROCEDURE

Before testing, secure the motor, pulley and belt. Make sure the sensor sees each slot/mark once per pass. Start with the belt unloaded and a low PWM command. Open the Arduino Serial Monitor at 9600 baud and observe the target RPM, measured RPM, PWM and error columns.

Calibration steps

1. Count the number of sensor pulses produced by one full shaft revolution and set PULSES_PER_REV correctly.
2. Confirm the measured RPM is plausible at a low, steady speed. If it is consistently off by a fixed factor, re-check pulses per revolution and sensor alignment.
3. Set TARGET_RPM to a safe speed for the belt.
4. Observe whether PWM rises when measured RPM is below target and falls when it is above target.
5. Add a small, safe test load and record how quickly the speed returns toward the target.
6. Tune KP and KI in small increments. If the speed oscillates, reduce gains; if response is too slow, adjust gradually.
7. Save actual serial readings and screenshots from the simulator or hardware for submission.

Test matrix — fill with actual observations

Test	Condition	Expected behavior	Actual reading / result
1	Belt unloaded; target 60 RPM	Measured speed approaches target; PWM settles.	_____
2	Add a small load	Speed may dip initially; controller increases PWM to recover.	_____
3	Remove the load	Speed may rise; controller reduces PWM.	_____
4	Change target setpoint	Measured speed moves toward the new target.	_____
5	Block/lose sensor pulses briefly	Measured RPM reads low; inspect response and ensure the mechanism is stopped safely if needed.	_____

10. RESULTS AND DISCUSSION

The controller is designed to compare the target speed with the measured speed and change the PWM command accordingly. The expected qualitative response is an increase in PWM when the belt slows under load and a decrease when the speed rises above the target. Actual settling time, speed error, and stability depend on motor characteristics, belt friction, sensor pulse count, pulley diameter, supply voltage, load, and PI gains.

Do not submit the expected behavior as if it were measured data. Enter actual readings in the test matrix after running the circuit in a simulator or on the assembled hardware. Attach simulator screenshots or serial-monitor evidence if required by your instructor.

11. ADVANTAGES, LIMITATIONS AND APPLICATIONS

Advantages

• Automatically corrects speed deviations instead of using a fixed PWM only.
• Low-cost implementation with commonly available embedded components.
• Serial output makes it easier to monitor and tune the controller.

Limitations

• Accuracy depends on correct sensor alignment and pulses-per-revolution calibration.
• The L298N has voltage losses and is less efficient than modern MOSFET drivers.
• PI gains must be tuned for the actual motor and mechanical load.
• A lost or noisy sensor signal can cause incorrect speed estimates; additional fault handling is recommended for a production system.

Applications